

Laboratory 3: Search and A*

Using an LLM as an Engineering Assistant

Eshan Saikia
2023A7PS1024G

The accompanying `search.ipynb` contains the executable implementation and validation evidence. This report records the search formulation, design, LLM use, results, heuristic investigation, and reflections.

1 Search Problem Specification and Design

The warehouse supplied in the laboratory is:

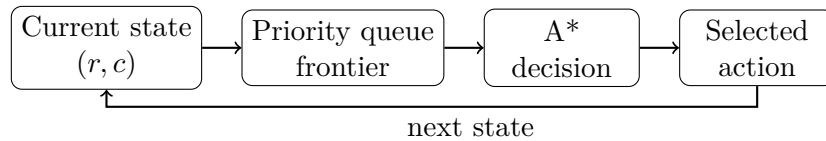
```
#####  
#S....#.....#  
#.###.#.#####.#  
#...#.#.....#.#  
###.#.#####.#.#  
#...#.....#.#  
#.#####.#.#  
#.....#G#  
#####
```

The search problem is $P = (S, A, T, s_0, G, c)$: states are reachable grid positions (r, c) ; actions are Up, Down, Left, and Right; transitions move one cell when the destination is inside the grid and not an obstacle; s_0 is S ; G is the goal; and each movement costs 1. The problem is deterministic, and a solution is any legal action sequence that reaches G .

The agent uses

$$f(n) = g(n) + h(n), \quad h(n) = |r - r_G| + |c - c_G|.$$

A priority queue stores the frontier, while best-known path costs and parent/action records support duplicate handling and path reconstruction.



2 LLM-Assisted Implementation

The implementation prompt was:

I am implementing a simple warehouse search agent in Python. The environment is the supplied ASCII grid. **S** is the start, **G** is the goal, **#** is an obstacle, and **.** is free. The robot moves Up, Down, Left, or Right and every move costs 1. Implement A* search using Manhattan distance $h(n) = |r - r_G| + |c - c_G|$. Represent positions as states, maintain a priority-queue frontier, track $g(n)$, $h(n)$, and $f(n) = g(n) + h(n)$, avoid unnecessary repeated expansion, store parent/action information, reconstruct the path, and report solution found, path, path length, and states expanded. Keep it dependency-free and easy to inspect.

The generated A* search logic worked on first execution. I retained it after source inspection and added validation assertions, the four required tests, a BFS implementation for comparison, and the required heuristic variants. These additions did not change the search formulation.

3 Experimental Results

For the original warehouse, A* found a path of length 40 and expanded 64 states. A compact description of the returned action sequence is

$$R^4 D^4 R^8 U^2 L^6 U^2 R^8 D^6.$$

The complete path is stored in the notebook.

Test	Found	Path length	States expanded
Original warehouse	Yes	40	64
Trivial one-step case	Yes	1	2
No-solution case	No	—	9
Alternative-path case	Yes	4	5

The notebook independently verifies legal transitions and obstacle avoidance. In the alternative-path test, A* and BFS both returned a shortest path of length 4.

4 A* Inspection and BFS Comparison

Concept	Representation	Purpose
State	(r, c)	Robot position
Action	Four directions	Transition choice
Transition	Position + displacement	Successor generation
Goal test	Current = goal	Termination
$g(n)$	Best path cost	Cost from start
$h(n)$	Manhattan distance	Estimate to goal
$f(n)$	$g + h$	Frontier priority
Frontier	Priority queue	Next state selection
Visited records	Best-cost/expansion maps	Duplicate control
Path reconstruction	Parent/action maps	Recover solution

Measure	BFS	A*
Solution found	Yes	Yes
Path length	40	40
States expanded	64	64

Neither method expanded fewer states on this particular map and action order. A* can reduce exploration when its heuristic provides stronger guidance.

5 Heuristic Investigation

Heuristic	Solution	Path length	States expanded
$h(n) = 0$	Yes	40	64
Manhattan	Yes	40	64
Euclidean	Yes	40	64
$2 \times$ Manhattan	Yes	40	68

With $h = 0$, A* reduces to uniform-cost search and behaves like BFS because every movement has unit cost. Manhattan distance is admissible and natural for horizontal/vertical movement. Euclidean distance is also admissible here but did not alter the expansion count. Multiplying Manhattan distance by 2 can overestimate the remaining cost and is non-admissible in general; this map still yielded the optimal length-40 path, but without a shortest-path guarantee.

6 Task 7 - LLM Evaluation

Question	Answer
Correct immediately?	State representation, four actions, Manhattan heuristic, $g/h/f$, priority queue, goal test, and path reconstruction.
Bugs/design problems?	No functional bug appeared in the first execution.
How discovered?	Source inspection followed by known-answer tests.
Unfamiliar terminology?	No; the data structures and A* quantities were standard and directly inspected.
Modified?	Yes: validation tests, assertions, BFS comparison, heuristic variants, and stale-entry handling.
Most useful tests?	The inaccessible-goal and alternative-path tests.
Trust without testing?	No; working output does not establish correct search behaviour.
New understanding?	$g(n)$ captures cost already incurred while $h(n)$ changes the frontier order using information about the goal.

7 Final Reflection

1. **Problem formulation.** It fixes states, actions, transitions, the initial state, goal, and costs independently of implementation and provides a verification specification.
2. **A* as informed search.** A* uses $h(n)$, an estimate of remaining cost, together with $g(n)$ through $f(n) = g(n) + h(n)$.
3. **Heuristic choice.** The heuristic changes the expansion order; admissibility matters when shortest-path guarantees are required.
4. **LLM contribution.** The LLM translated the specified A* design into Python and supplied an implementation starting point; the resulting behaviour was then independently tested.
5. **Risk of untested code.** Generated code can run while still mishandling obstacles, priorities, duplicate states, goal recognition, or path reconstruction.